Name: Deras Pangestu Gusti (08)
Class: SIB 1G
NIM: 254107060114

JOBSHEET - 6

SORTING (BUBBLE, SELECTION, DAN INSERTION SORT)

6.1 Course Leaning Outcome

After completing this practicum, students should be able to:

a. Create sorting algorithms for bubble sort, selection sort, and insertion sort.

b. Implement sorting algorithms (bubble sort, selection sort, and insertion sort) in a program.

6.2 Experiment 1 - Implementing Sorting Using Objects

Duration : 60 minutes

6.2.1 Experiment Steps

A. SORTING – BUBBLE SORT

1. Create a new folder named **week05**.

2. Create a class **Sorting<Attendance Number>**, then add the following attributes:

```
public class Sorting {
    int[] data;
    int size;
}
```

3. Create a constructor with parameters data[] and jmlDat.

```
public Sorting(int[] data) {
    this.data = data;
    this.size = data.length;
}
```

4. Create a **bubbleSort** method of type void and declare its contents using the Bubble Sort algorithm.

```
public void bubbleSort() {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (data[j] > data[j + 1]) {
                int temp = data[j];
                data[j] = data[j + 1];
                data[j + 1] = temp;
            }
        }
    }
}
```

5. Create a **print** method of type void and declare the content of the method.

```
public void print() {
    for (int num : data) {
        System.out.print(num + " ");
    }
    System.out.println();
}
```

6. Create a class **SortingMain<Attendance Number>**, then declare an array named **a[]** and fill it.

```
int[] a = {34, 7, 23, 32, 5, 62};
```

7. Create a new object named **sorting1**, which is an instantiation of the Sorting class, then fill in its parameters.

```
Sorting sorting = new Sorting(a);
```

8. Call the **bubbleSort** and **print** methods.

```
System.out.println(x:"Original array:");
sorting.print();
sorting.bubbleSort();
System.out.println(x:"Sorted array (Bubble Sort):");
sorting.print();
```

9. Run the program and observe the results.

```java
package com.jobsheet6;

public class Sorting8
{
    int[] data;
    int size;

    public Sorting8 (int[] data)
    {
        this.data = data;
        this.size = data.length;
    }

    public void bubbleSort()
    {
        for (int i = 0; i < size - 1; i++)
        {
            for (int j = 0; j < size - i - 1; j++)
            {
                if (data[j] > data[j + 1])
                {
                    int temp = data[j];
                    data[j] = data[j + 1];
                    data[j + 1] = temp;
                }
            }
        }
    }

    public void print()
    {
        for (int num : data)
        {
            System.out.print(num + " ");
        }
        System.out.println();
    }
}
```

```java
package com.jobsheet6;

public class SortinngMain
{

    public static void main(String[] args)
    {
        int[] a = {34, 7, 23, 32, 5, 62};

        Sorting8 sorting = new Sorting8(a);

        System.out.println(x: "Original Array:");
        sorting.print();
        sorting.bubbleSort();
        System.out.println(x: "Sorted array (Bubble Sort):");
        sorting.print();
    }
}
```

6.2.2 Verification of Experiment Results

```
Original array:
34 7 23 32 5 62
Sorted array (Bubble Sort):
5 7 23 32 34 62
```

B. SORTING – SELECTION SORT

1. In the **Sorting<Attendance Number>** class created in the previous step, add the **selectionSort** method that implements sorting using the selection sort algorithm.

```java
public void selectionSort() {
    for (int i = 0; i < size - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < size; j++) {
            if (data[j] < data[minIndex]) {
                minIndex = j;
            }
        }
        int temp = data[minIndex];
        data[minIndex] = data[i];
        data[i] = temp;
    }
}
```

2. Declare an array named **b[]** in the **SortingMain<Attendance Number>** class, then fill it.

```
int[] b = {30, 20, 2, 8, 14};
```

3. Create a new object named **sorting2**, which is an instantiation of the **Sorting** class, then assign in its parameters.

```
Sorting sorting2 = new Sorting(b);
```

4. Call the **selectionSort** and **print** methods.

```
System.out.println(x:"Original array:");
sorting2.print();
sorting2.selectionSort();
System.out.println(x:"Sorted array (Selection Sort):");
sorting2.print();
```

5. Run the program and observe the results.

```java
package com.jobsheet6;

public class Sorting8
{
    int[] data;
    int size;
    public Sorting8 (int[] data)
    {
        this.data = data;
        this.size = data.length;
    }

    public void bubbleSort()
    {
        for (int i = 0; i < size - 1; i++)
        {
            for (int j = 0; j < size - i - 1; j++)
            {
                if (data[j] > data[j + 1])
                {
                    int temp = data[j];
                    data[j] = data[j + 1];
                    data[j + 1] = temp;
                }
            }
        }
    }

    public void print()
    {
        for (int num : data)
        {
            System.out.print(num + " ");
        }
        System.out.println();
    }

    public void selectionSort()
    {
        for (int i = 0; i < size - 1; i++)
        {
            int minIndex = i;
            for (int j = 0; j < size; j++)
            {
                if (data[j] < data[minIndex])
                {
                    minIndex = j;
                }
            }
            int temp = data[minIndex];
            data[minIndex] = data[i];
            data[i] = temp;
        }
    }
}
```

```java
package com.jobsheet6;

public class SortinngMain
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        int[] a = {34, 7, 23, 32, 5, 62};
        int[] b = {30, 20, 2, 8, 14};

        Sorting8 sorting = new Sorting8(a);
        Sorting8 sorting2 = new Sorting8(b);

        System.out.println(x: "Original Array:");
        sorting.print();
        sorting.bubbleSort();
        System.out.println(x: "Sorted array (Bubble Sort):"
        sorting.print();

        System.out.println(x: "Original Array:");
        sorting2.print();
        sorting2.selectionSort();
        System.out.println(x: "Sorted array (Selection Sort
        sorting2.print();
    }
}
```

```
Original Array:
34 7 23 32 5 62
Sorted array (Bubble Sort):
5 7 23 32 34 62
Original Array:
30 20 2 8 14
Sorted array (Selection Sort):
20 30 8 2 14
```

6.2.3 Verification of Experiment Results

```
Original array:
30 20 2 8 14
Sorted array (Selection Sort):
2 8 14 20 30
```

C. SORTING – INSERTION SORT

1. In the **Sorting<Attendance Number>** class created in the previous step, add the **insertionSort** method that implements sorting using the insertion sort algorithm.

```java
public void insertionSort() {
    for (int i = 1; i < size; i++) {
        int key = data[i];
        int j = i - 1;
        while (j >= 0 && data[j] > key) {
            data[j + 1] = data[j];
            j--;
        }
        data[j + 1] = key;
    }
}
```

2. Declare an array named **c[]** in the **SortingMain<Attendance Number>** class, then fill it.

```java
int c[]= {40, 10, 4, 9, 3};
```

3. Create a new object named **dataurut3**, which is an instantiation of the Sorting class, then fill in its parameters.

```java
Sorting sorting3 = new Sorting(c);
```

4. Call the **insertionSort** and **display** methods.

```java
System.out.println(x:"Original array:");
sorting3.print();
sorting3.insertionSort();
System.out.println(x:"Sorted array (Insertion Sort):");
sorting3.print();
```

5. Run the program and observe the results!

```java
package com.jobsheet6;

public class Sorting8
{
    int[] data;
    int size;

    public Sorting8 (int[] data)
    {
        this.data = data;
        this.size = data.length;
    }

    public void bubbleSort()
    {
        for (int i = 0; i < size - 1; i++)
        {
            for (int j = 0; j < size - i - 1; j++)
            {
                if (data[j] > data[j + 1])
                {
                    int temp = data[j];
                    data[j] = data[j + 1];
                    data[j + 1] = temp;
                }
            }
        }
    }

    public void print()
    {
        for (int num : data)
        {
            System.out.print(num + " ");
        }
        System.out.println();
    }

```

```
    public void selectionSort()
    {
        for (int i = 0; i < size - 1; i++)
        {
            int minIndex = i;
            for (int j = 0; j < size; j++)
            {
                if (data[j] < data[minIndex])
                {
                    minIndex = j;
                }
            }
            int temp = data[minIndex];
            data[minIndex] = data[i];
            data[i] = temp;
        }
    }

    public void insertionSort()
    {
        for (int i = 1; i < size; i++)
        {
            int key = data[i];
            int j = i - 1;
            while (j >= 0 && data[j] > key)
            {
                data[j + 1] = data[j];
                j--;
            }
            data[j + 1] = key;
        }
    }
}
```

```java
package com.jobsheet6;

public class SortinngMain
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        int[] a = {34, 7, 23, 32, 5, 62};
        int[] b = {30, 20, 2, 8, 14};
        int c[] = {40, 10, 4, 9, 3};

        Sorting8 sorting = new Sorting8(a);
        Sorting8 sorting2 = new Sorting8(b);
        Sorting8 sorting3 = new Sorting8(c);

        System.out.println(x: "Original Array:");
        sorting.print();
        sorting.bubbleSort();
        System.out.println(x: "Sorted array (Bubble Sort):");
        sorting.print();

        System.out.println(x: "Original Array:");
        sorting2.print();
        sorting2.selectionSort();
        System.out.println(x: "Sorted array (Selection Sort):");
        sorting2.print();

        System.out.println(x: "Original Array:");
        sorting3.print();
        sorting3.insertionSort();
        System.out.println(x: "Sorted array (Insertion Sort):");
        sorting3.print();
    }
}
```

```
Original Array:
40 10 4 9 3
Sorted array (Insertion Sort):
3 4 9 10 40
```

6.2.4 Verification of Experiment Results

```
Original array:
40 10 4 9 3
Sorted array (Insertion Sort):
3 4 9 10 40
```

6.2.5 Questions!

1. Explain the function of the following program code:

```
if (data[j] > data[j + 1]) {
    int temp = data[j];
    data[j] = data[j + 1];
    data[j + 1] = temp;
}
```

It compares two adjacent elements in the array. If the current element data[j] is greater than the next element data[j + 1], a swap is performed using a temporary variable temp.

2. Show the program code that implements the minimum value search algorithm in selection sort!

```
int minIndex = i;
for (int j = 0; j < size; j++)
{
    if (data[j] < data[minIndex])
    {
        minIndex = j;
    }
}
int temp = data[minIndex];
data[minIndex] = data[i];
data[i] = temp;
```

3. In insertion sort, explain the purpose of the condition in the loop.

```
while (j >= 0 && data[j] > key)
```

   a. *j >= 0 → Prevents the index from going out of bounds (below index 0).*
   b. *data[j] > key → Keeps shifting elements to the right as long as they are larger than the key (the element being inserted).*

4. In insertion sort, what is the purpose of the given command?

```
data[j + 1] = data[j];
```

This command shifts the element at position j one position to the right (to position j + 1), making room for the key to be inserted in its correct sorted position. It simply copies it one slot forward.

6.3 Experiment 2- Sorting Using an Array of Objects

Duration: 45 minutes

6.3.1 <u>Experiment Steps - Sorting Student Data Based on GPA (Bubble Sort)</u>

Observe the **Student** class diagram below! This diagram will serve as a reference for creating the **Student** class.

Student

nim: String name: String studentClass: String gpa: double
Student () Student (nm: String, name: String, clsNm: String, gpa: double) print(): void

The **Student** class will be used to create student objects that will be inserted into an array. It has a default constructor, a parameterized constructor, and a function print() to show all attributes data.

TopStudents
listStudent: Student[5] idx: int
add(std: Student): void print(): void bubbleSort(): void

This class performs operations on the array of student objects, such as adding, displaying, and sorting students using the Bubble Sort algorithm based on GPA.

6.3.2 Experiment Steps

1. Create a class named **Student<Attendance Number>**.
2. Complete the Student class with the following code!

```java
public class Student {
    String nim;
    String name;
    String studentClass;
    double gpa;

    public Student() {}

    public Student(String nim, String name, String studentClass, double gpa) {
        this.nim = nim;
        this.name = name;
        this.studentClass = studentClass;
        this.gpa = gpa;
    }

    public void print() {
        System.out.println("NIM: " + nim + ", Name: " + name + ", Class: " + studentClass + ", GPA: " + gpa);
    }
}
```

3. Create a class **TopStudents<Attendance Number>.**

```java
public class TopStudent {
    Student[] listStudents;
    int idx;
}
```

4. Add a method **add()** in this class to insert student objects into the **listStudents** attribute.

```java
public void add(Student student) {
    if (idx < listStudents.length) {
        listStudents[idx] = student;
        idx++;
    } else {
        System.out.println(x:"List is full!");
    }
}
```

5. Add a method **print()** in this class to display all students.

```java
public void print() {
    for (int i = 0; i < idx; i++) {
        listStudents[i].print();
    }
}
```

6. Add a method **bubbleSort()** in this class!

```java
public void bubbleSort() {
    for (int i = 0; i < idx - 1; i++) {
        for (int j = 0; j < idx - i - 1; j++) {
            if (listStudents[j].gpa < listStudents[j + 1].gpa) {
                Student temp = listStudents[j];
                listStudents[j] = listStudents[j + 1];
                listStudents[j + 1] = temp;
            }
        }
    }
}
```

7. Create a class **StudentDemo<Attendance Number>**, instantiate a **TopStudents** object, create 5 student objects, and add them using the **add** function. Call **print(),** then sort the data using **bubbleSort(),** and finally call **print()** again.

```java
public static void main(String[] args) {
    TopStudent topStudents = new TopStudent(size:5);

    // Adding student data
    topStudents.add(new Student(nim:"2201", name:"Alice", studentClass:"A", gpa:3.9));
    topStudents.add(new Student(nim:"2202", name:"Bob", studentClass:"B", gpa:3.7));
    topStudents.add(new Student(nim:"2203", name:"Charlie", studentClass:"C", gpa:3.8));
    topStudents.add(new Student(nim:"2204", name:"David", studentClass:"D", gpa:3.6));
    topStudents.add(new Student(nim:"2205", name:"Eve", studentClass:"E", gpa:4.0));

    // Display original list
    System.out.println(x:"Original student list:");
    topStudents.print();

    // Sorting students by GPA using Bubble Sort
    topStudents.bubbleSort();
    System.out.println(x:"Sorted student list (by GPA, descending):");
    topStudents.print();
}
```

6.3.3 Verification of Experiment Results

```
Original student list:
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2204, Name: David, Class: D, GPA: 3.6
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
Sorted student list (by GPA, descending):
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2204, Name: David, Class: D, GPA: 3.6
```

```java
package com.jobsheet6;

public class Student8
{
    String nim;
    String name;
    String studentClass;
    double gpa;

    public Student8() {}

    public Student8 (String nim, String name, String studentClass, double gpa)
    {
        this.nim = nim;
        this.name = name;
        this.studentClass = studentClass;
        this.gpa = gpa;
    }

    public void print()
    {
        System.out.println("NIM: " + nim + ", Name: " + name + ", Class: " + studentClass + ", GPA: " + gpa);
    }
}
```

```java
package com.jobsheet6;

public class TopStudents8
{
    Student8[] listStudents;
    int idx;

    public void add (Student8 student)
    {
        if (idx < listStudents.length)
        {
            listStudents[idx] = student;
            idx++;
        }
        else
        {
            System.out.println(x: "List is full");
        }
    }

    public void print()
    {
        for (int i = 0; i < idx; i++)
        {
            listStudents[i].print();
        }
    }

    public void bubbleSort()
    {
        for (int i = 0; i < idx - 1; i++)
        {
            for (int j = 0; j < idx - i - 1; j++)
            {
                if (listStudents[j].gpa < listStudents[j + 1].gpa)
                {
                    Student8 temp = listStudents[j];
                    listStudents[j] = listStudents[j + 1];
                    listStudents[j + 1] = temp;
                }
            }
        }
    }
}
```

```java
package com.jobsheet6;

public class StudentDemo8
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        TopStudents8 topStudents = new TopStudents8();
        topStudents.listStudents = new Student8[5];

        topStudents.add(new Student8(nim: "2201", name: "Alice", studentClass: "A", gpa: 3.9));
        topStudents.add(new Student8(nim: "2202", name: "Bob", studentClass: "B", gpa: 3.7));
        topStudents.add(new Student8(nim: "2203", name: "Charlie", studentClass: "C", gpa: 3.8));
        topStudents.add(new Student8(nim: "2204", name: "David", studentClass: "D", gpa: 3.6));
        topStudents.add(new Student8(nim: "2205", name: "Eve", studentClass: "E", gpa: 4.0));

        System.out.println(x: "Original student list:");
        topStudents.print();

        topStudents.bubbleSort();
        System.out.println(x: "Sorted student list (by GPA, descending):");
        topStudents.print();
    }
}
```

```
Original student list:
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2204, Name: David, Class: D, GPA: 3.6
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
Sorted student list (by GPA, descending):
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2204, Name: David, Class: D, GPA: 3.6
```

6.3.4 Questions!

1. From the following code snippet, answer question a-c:

```java
for (int i = 0; i < idx - 1; i++) {
    for (int j = 0; j < idx - i - 1; j++) {
```

   a. Why is the condition in the **bubbleSort()** loop **i < idx - 1**?

      The outer loop controls the number of passes through the array. After each pass, the largest unsorted element is guaranteed to be placed at its correct position at the end.

   b. Why is the condition in the **bubbleSort()** loop **j < idx - i - 1**?

After each pass i, the last i elements are already sorted and in their final positions. There is no need to compare those elements again.

c. If the number of data in **listStudents** is 50, how many times will the **i** loop execute?

How many stages of Bubble Sort will be performed?

- *Number of i loop executions: idx - 1 = 50 - 1 = 49 times*
- *Number of Bubble Sort stages (total comparisons): 49 + 48 + 47 + … + 1 =* $\frac{49 \times 50}{2}$ *= 1,225 comparisons*

2. Modify the above program to allow dynamic student data input (from the keyboard) consisting of nim, name, studentClass, and gpa.

```java
package com.jobsheet6;
import java.util.Scanner;

public class StudentDemo8Modify
{
    Run main | Debug main
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);

        System.out.print(s: "Enter the number of students: ");
        int n = sc.nextInt();
        sc.nextLine(); // consume newline

        TopStudents8 topStudents = new TopStudents8();
        topStudents.listStudents = new Student8[n];

        for (int i = 0; i < n; i++)
        {
            System.out.println("--- Student " + (i + 1) + " ---");

            System.out.print(s: "NIM: ");
            String nim = sc.nextLine();

            System.out.print(s: "Name: ");
            String name = sc.nextLine();

            System.out.print(s: "Class: ");
            String studentClass = sc.nextLine();

            System.out.print(s: "GPA: ");
            double gpa = sc.nextDouble();
            sc.nextLine();

            topStudents.add(new Student8(nim, name, studentClass, gpa));
        }

        System.out.println(x: "Original student list:");
        topStudents.print();
        topStudents.bubbleSort();
        System.out.println(x: "Sorted student list (by GPA, descending):");
        topStudents.print();
        sc.close();
    }
}
```

6.3.5 Sorting Student Data Based on GPA (Selection Sort)

Duration: 30 minutes

If we previously sorted students based on GPA using Bubble Sort in descending order, we will now add a function to sort using Selection Sort in ascending order.

6.3.7 Experiment Steps

1. Modify **TopStudents** to include a **selectionSort()** method.

```
public void selectionSort() {
    for (int i = 0; i < idx - 1; i++) {
        int minIndex = i;
        for (int j = i + 1; j < idx; j++) {
            if (listStudents[j].gpa < listStudents[minIndex].gpa) {
                minIndex = j;
            }
        }
        // Swap the found minimum element
        Student temp = listStudents[minIndex];
        listStudents[minIndex] = listStudents[i];
        listStudents[i] = temp;
    }
}
```

2. Modify **StudentDemo** and add program lines in **main()** to call **selectionSort()** and **print().**

```
topStudents.selectionSort();
System.out.println(x:"Sorted student list (by GPA, ascending) using Selection Sort:");
topStudents.print();
```

3. Run **StudentDemo** and observe if the data is sorted in ascending order.

```java
package com.jobsheet6;

public class TopStudents8
{
    Student8[] listStudents;
    int idx;

    public void add (Student8 student)
    {
        if (idx < listStudents.length)
        {
            listStudents[idx] = student;
            idx++;
        }
        else
        {
            System.out.println(x: "List is full");
        }
    }

    public void print()
    {
        for (int i = 0; i < idx; i++)
        {
            listStudents[i].print();
        }
    }

    public void bubbleSort()
    {
        for (int i = 0; i < idx - 1; i++)
        {
            for (int j = 0; j < idx - i - 1; j++)
            {
                if (listStudents[j].gpa < listStudents[j + 1].gpa)
                {
                    Student8 temp = listStudents[j];
                    listStudents[j] = listStudents[j + 1];
                    listStudents[j + 1] = temp;
                }
            }
        }
    }

```

```java
    public void selectionSort()
    {
        for (int i = 0; i < idx - 1; i++)
        {
            int minIndex = i;
            for (int j = i + 1; j < idx; j++)
            {
                if (listStudents[j].gpa < listStudents[minIndex].gpa)
                {
                    minIndex = j;
                }
            }
            Student8 temp = listStudents[minIndex];
            listStudents[minIndex] = listStudents[i];
            listStudents[i] = temp;
        }
    }
}
```

```java
package com.jobsheet6;

public class StudentDemo8
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        TopStudents8 topStudents = new TopStudents8();
        topStudents.listStudents = new Student8[5];

        topStudents.add(new Student8(nim: "2201", name: "Alice", studentClass: "A", gpa: 3.9));
        topStudents.add(new Student8(nim: "2202", name: "Bob", studentClass: "B", gpa: 3.7));
        topStudents.add(new Student8(nim: "2203", name: "Charlie", studentClass: "C", gpa: 3.8
        topStudents.add(new Student8(nim: "2204", name: "David", studentClass: "D", gpa: 3.6));
        topStudents.add(new Student8(nim: "2205", name: "Eve", studentClass: "E", gpa: 4.0));

        System.out.println(x: "Original student list:");
        topStudents.print();

        topStudents.bubbleSort();
        System.out.println(x: "Sorted student list (by GPA, descending):");
        topStudents.print();

        topStudents.selectionSort();
        System.out.println(x: "Sorted student list (by GPA, ascending), using Selection Sort:
        topStudents.print();
    }
}
```

```
Sorted student list (by GPA, ascending), using Selection Sort:
NIM: 2204, Name: David, Class: D, GPA: 3.6
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
```

6.3.8 Verification of Experiment Results

Make sure the output of your code is like the following image.

```
Sorted student list (by GPA, ascending) using Selection Sort:
NIM: 2204, Name: David, Class: D, GPA: 3.6
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
```

6.3.9 Questions!

Explain the following code snippet in the correlation with the selection sort!

```
int minIndex = i;
for (int j = i + 1; j < idx; j++) {
    if (listStudents[j].gpa < listStudents[minIndex].gpa) {
        minIndex = j;
    }
}
```

It sorts objects by comparing a specific attribute (gpa). Each pass guarantees that the student with the lowest GPA among the remaining unsorted students is moved to its correct position, resulting in a list sorted by GPA in ascending order.

6.3.10 Sorting Student Data Based on GPA Using Insertion Sort

Duration: 30 minutes

6.3.11 Experiment Steps

1. Modify **TopStudents** to include an **insertionSort()** method.

```
public void insertionSort() {
    for (int i = 1; i < idx; i++) {
        Student temp = listStudents[i];
        int j = i;

        // Move elements that have bigger GPA to the right
        while (j > 0 && listStudents[j-1].gpa > temp.gpa) {
            listStudents[j] = listStudents[j-1];
            j--;
        }
        listStudents[j] = temp;
    }
}
```

2. Modify **StudentDemo** and add program lines in **main()** to call **insertionSort()** and **print().**

```
topStudents.insertionSort();
System.out.println(x:"Sorted student list (by GPA, ascending) using Insertion Sort:");
topStudents.print();
```

3. Run **StudentDemo** and observe if the data is sorted in ascending order.

```java
package com.jobsheet6;

public class TopStudents8
{
    Student8[] listStudents;
    int idx;

    public void add (Student8 student)
    {
        if (idx < listStudents.length)
        {
            listStudents[idx] = student;
            idx++;
        }
        else
        {
            System.out.println(x: "List is full");
        }
    }

    public void print()
    {
        for (int i = 0; i < idx; i++)
        {
            listStudents[i].print();
        }
    }

    public void bubbleSort()
    {
        for (int i = 0; i < idx - 1; i++)
        {
            for (int j = 0; j < idx - i - 1; j++)
            {
                if (listStudents[j].gpa < listStudents[j + 1].gpa)
                {
                    Student8 temp = listStudents[j];
                    listStudents[j] = listStudents[j + 1];
                    listStudents[j + 1] = temp;
                }
            }
        }
    }

```

```
    public void selectionSort()
    {
        for (int i = 0; i < idx - 1; i++)
        {
            int minIndex = i;
            for (int j = i + 1; j < idx; j++)
            {
                if (listStudents[j].gpa < listStudents[minIndex].gpa)
                {
                    minIndex = j;
                }
            }
            Student8 temp = listStudents[minIndex];
            listStudents[minIndex] = listStudents[i];
            listStudents[i] = temp;
        }
    }

    public void insertionSort()
    {
        for (int i = 1; i < idx; i++)
        {
            Student8 temp = listStudents[i];
            int j = i;

            while (j > 0 && listStudents[j - 1].gpa > temp.gpa)
            {
                listStudents[j] = listStudents[j - 1];
                j--;
            }
            listStudents[j] = temp;
        }
    }
}
```

```java
package com.jobsheet6;

public class StudentDemo8
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        TopStudents8 topStudents = new TopStudents8();
        topStudents.listStudents = new Student8[5];

        topStudents.add(new Student8(nim: "2201", name: "Alice", studentClass: "A", gpa: 3.9));
        topStudents.add(new Student8(nim: "2202", name: "Bob", studentClass: "B", gpa: 3.7));
        topStudents.add(new Student8(nim: "2203", name: "Charlie", studentClass: "C", gpa: 3.8));
        topStudents.add(new Student8(nim: "2204", name: "David", studentClass: "D", gpa: 3.6));
        topStudents.add(new Student8(nim: "2205", name: "Eve", studentClass: "E", gpa: 4.0));

        System.out.println(x: "Original student list:");
        topStudents.print();

        topStudents.bubbleSort();
        System.out.println(x: "Sorted student list (by GPA, descending):");
        topStudents.print();

        topStudents.selectionSort();
        System.out.println(x: "Sorted student list (by GPA, ascending), using Selection Sort:");
        topStudents.print();

        topStudents.insertionSort();
        System.out.println(x: "Sorted student list (by GPA, ascending), using Insertion Sort:");
        topStudents.print();
    }
}
```

```
Sorted student list (by GPA, ascending), using Insertion Sort:
NIM: 2204, Name: David, Class: D, GPA: 3.6
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
```

6.3.12 Verification of Experiment Results

Make sure the output of your code is like the following image.

```
Sorted student list (by GPA, ascending) using Insertion Sort:
NIM: 2204, Name: David, Class: D, GPA: 3.6
NIM: 2202, Name: Bob, Class: B, GPA: 3.7
NIM: 2203, Name: Charlie, Class: C, GPA: 3.8
NIM: 2201, Name: Alice, Class: A, GPA: 3.9
NIM: 2205, Name: Eve, Class: E, GPA: 4.0
```

6.3.13 Question

Modify the **insertionSort()** method so that it can perform a descending sorting!

```
    public void insertionSort()
    {
        for (int i = 1; i < idx; i++)
        {
            Student8 temp = listStudents[i];
            int j = i;

            while (j > 0 && listStudents[j - 1].gpa < temp.gpa)
            {
                listStudents[j] = listStudents[j - 1];
                j--;
            }
            listStudents[j] = temp;
        }
    }
}
```

6.4 Assignment

Duration: 45 minutes

Observe the following class diagram and create a program that performs the following menu options:

1. Add Data - Add lecturer data.
2. Display Data - Show all lecturer data.
3. Sort ASC - Sort lecturers by age from youngest to oldest using Bubble Sort.
4. Sort DSC - Sort lecturers by age from oldest to youngest using Selection Sort or Insertion Sort.

Lecturer
id: String name: String gender: Boolean age: int
Lecturer(id: String, name: String, gender: Boolean, age: int) print(): void

LecturerData
lecturerData: Lecturer[10] idx: int
add(dsn: Dosen): void print(): void

sortingASC(): void sortingDSC():void

```java
package com.jobsheet6;

public class Lecturer
{
    String id;
    String name;
    Boolean gender;
    int age;

    public Lecturer() {}

    public Lecturer(String id, String name, Boolean gender, int age)
    {
        this.id = id;
        this.name = name;
        this.gender = gender;
        this.age = age;
    }

    public void print()
    {
        String genderStr = (gender) ? "Male" : "Female";
        System.out.println("ID: " + id + ", Name: " + name +
                           ", Gender: " + genderStr + ", Age: " + age);
    }
}
```

```java
package com.jobsheet6;

public class LecturerData
{
    Lecturer[] lecturerData;
    int idx;

    public LecturerData()
    {
        lecturerData = new Lecturer[10];
        idx = 0;
    }

    public void add(Lecturer dsn)
    {
        if (idx < lecturerData.length)
        {
            lecturerData[idx] = dsn;
            idx++;
        }
        else
        {
            System.out.println(x: "List is full!");
        }
    }

    public void print()
    {
        if (idx == 0)
        {
            System.out.println(x: "No data available.");
            return;
        }
        for (int i = 0; i < idx; i++)
        {
            lecturerData[i].print();
        }
    }

```

```
    public void sortingASC()
    {
        for (int i = 0; i < idx - 1; i++)
        {
            for (int j = 0; j < idx - i - 1; j++)
            {
                if (lecturerData[j].age > lecturerData[j + 1].age)
                {
                    Lecturer temp = lecturerData[j];
                    lecturerData[j] = lecturerData[j + 1];
                    lecturerData[j + 1] = temp;
                }
            }
        }
    }

    public void sortingDSC()
    {
        for (int i = 1; i < idx; i++)
        {
            Lecturer temp = lecturerData[i];
            int j = i;

            while (j > 0 && lecturerData[j - 1].age < temp.age)
            {
                lecturerData[j] = lecturerData[j - 1];
                j--;
            }
            lecturerData[j] = temp;
        }
    }
}
```

Github:
https://github.com/guramedadar/Praktikum-Algoritma-Struktur-Data/tree/main/src/com/jobsheet6

```java
package com.jobsheet6;

import java.util.Scanner;

public class LecturerMain
{
    Run main | Debug main | Run | Debug
    public static void main(String[] args)
    {
        Scanner scanner = new Scanner(System.in);
        LecturerData data = new LecturerData();
        int choice;

        do
        {
            System.out.println(x: "\n========== LECTURER MENU ==========");
            System.out.println(x: "1. Add Data");
            System.out.println(x: "2. Display Data");
            System.out.println(x: "3. Sort ASC (Youngest to Oldest) - Bubble Sort");
            System.out.println(x: "4. Sort DSC (Oldest to Youngest) - Insertion Sort");
            System.out.println(x: "0. Exit");
            System.out.print(s: "Choose an option: ");
            choice = scanner.nextInt();
            scanner.nextLine(); // consume newline

```

```java
            switch (choice)
            {
                case 1:
                    System.out.println(x: "--- Add Lecturer ---");
                    System.out.print(s: "ID     : ");
                    String id = scanner.nextLine();

                    System.out.print(s: "Name   : ");
                    String name = scanner.nextLine();

                    System.out.print(s: "Gender (true = Male, false = Female): ");
                    Boolean gender = scanner.nextBoolean();
                    scanner.nextLine();

                    System.out.print(s: "Age    : ");
                    int age = scanner.nextInt();
                    scanner.nextLine();

                    data.add(new Lecturer(id, name, gender, age));
                    System.out.println(x: "Lecturer added successfully!");
                    break;

                case 2:
                    System.out.println(x: "--- Lecturer List ---");
                    data.print();
                    break;

                case 3:
                    data.sortingASC();
                    System.out.println(x: "--- Sorted by Age (ASC) using Bubble Sort ---");
                    data.print();
                    break;

                case 4:
                    data.sortingDSC();
                    System.out.println(x: "--- Sorted by Age (DSC) using Insertion Sort ---");
                    data.print();
                    break;

                case 0:
                    System.out.println(x: "Exiting programme. Goodbye!");
                    break;

                default:
                    System.out.println(x: "Invalid option. Please try again.");
            }

        } while (choice != 0);

        scanner.close();
    }
}
```

```
========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 2
--- Lecturer List ---
No data available.
```

```
========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 1
--- Add Lecturer ---
ID     : 001
Name   : Deras
Gender (true = Male, false = Female): true
Age    : 20
Lecturer added successfully!

========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 1
--- Add Lecturer ---
ID     : 002
Name   : Wendry
Gender (true = Male, false = Female): true
Age    : 19
Lecturer added successfully!

========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 1
--- Add Lecturer ---
ID     : 003
Name   : Yoga
Gender (true = Male, false = Female): true
Age    : 24
Lecturer added successfully!
```

```
========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 2
--- Lecturer List ---
ID: 001, Name: Deras, Gender: Male, Age: 20
ID: 002, Name: Wendry, Gender: Male, Age: 19
ID: 003, Name: Yoga, Gender: Male, Age: 24

========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 3
--- Sorted by Age (ASC) using Bubble Sort ---
ID: 002, Name: Wendry, Gender: Male, Age: 19
ID: 001, Name: Deras, Gender: Male, Age: 20
ID: 003, Name: Yoga, Gender: Male, Age: 24

========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 4
--- Sorted by Age (DSC) using Insertion Sort ---
ID: 003, Name: Yoga, Gender: Male, Age: 24
ID: 001, Name: Deras, Gender: Male, Age: 20
ID: 002, Name: Wendry, Gender: Male, Age: 19

========== LECTURER MENU ==========
1. Add Data
2. Display Data
3. Sort ASC (Youngest to Oldest) - Bubble Sort
4. Sort DSC (Oldest to Youngest) - Insertion Sort
0. Exit
Choose an option: 0
Exiting programme. Goodbye!
```